KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
POLITEKNIK NEGERI MALANG
JURUSAN TEKNOLOGI INFORMASI
Jl. Soekarno Hatta No. 9, Jatimulyo, Lowokwaru, Malang 65141
Telp. (0341) 404424 – 404425, Fax (0341) 404420
https://jti.polinema.ac.id

Mata Kuliah	:	**PEMROGRAMAN BERBASIS FRAMEWORK (PBF)**
Program Studi	:	D4 – Teknik Informatika
Semester	:	6

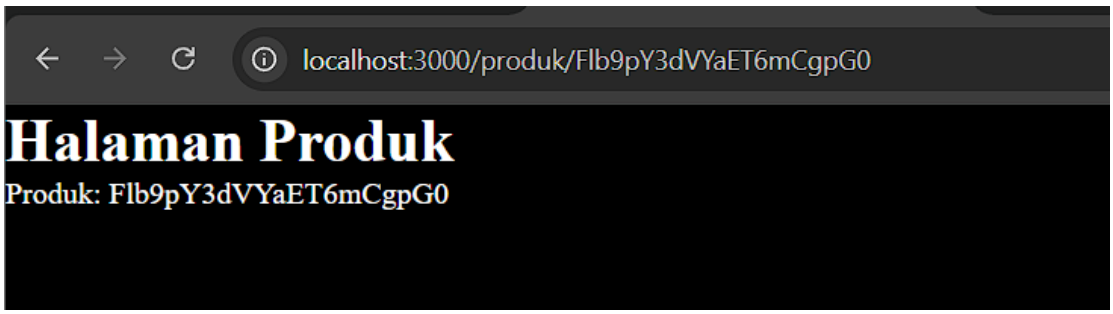
Kelas	:	TI-3B
NIM	:	2341720164
Nama	:	Raudhil Firdaus Naufal
Jobsheet Ke-	:	11

Laporan Jobsheet

Langkah 1 - Membuat Dynamic Route

Langkah	Jawaban/Deskripsi
1	Buka file pages/products/[product].tsx dan modfikasi sbb

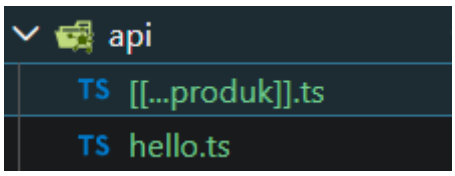
```
import styles from "../../produk/product.module.
scss";
import Link from "next/link";

type ProductType = {
  id: string;
  name: string;
  price: number;
  image: string;
  category: string;
};

const TampilanProduk = ({ products }: { products:
ProductType[] }) => {
  return (
    <div className={styles.produk}>
      <h1 className={styles.produk_title}>Daftar
      Produk</h1>

      <div className={styles.produk_content}>
        {products.length > 0 ? (
          <>
            {products.map((product: ProductType) => (
              <Link
                href={`/produk/${product.id}`}
                key={product.id}
                className={styles.
                produk_content_item}
              >
                <div className={styles.
                produk_content_item_image}>
                  <img
                    src={product.image}
                    alt={product.name}
                    width={200}
                  />
                </div>

                <h4 className={styles.
                produk_content_item_name}>
                  {product.name}
                </h4>
```

2	Jalankan browser http://localhost:3000/produk  Berhasil menampilkan info produk didalam setiap produk

Langkah 2 - Implementasi CSR (Client Rendering)

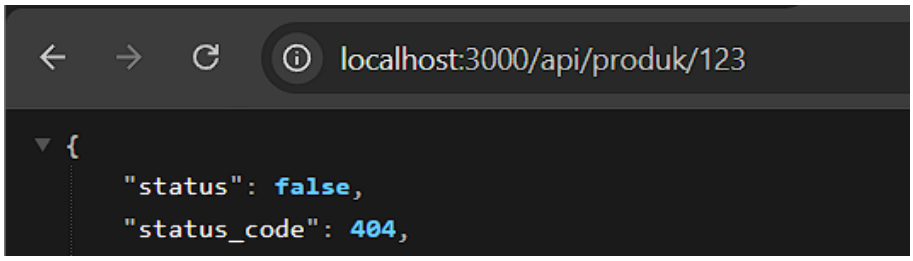
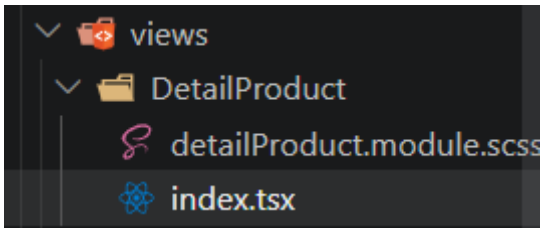
Langkah	Jawaban/Deskripsi
1	Modifikasi pada file [produk].tsx pada folder src/pages/produk/

```
import fetcher from "../utils/swr/fetcher";
import { useRouter } from "next/router";
import useSWR from "swr";

const HalamanProduk = () => {
  // const router = useRouter();
  // console.log(router);

  const { query } = useRouter();
  const { data, error, isLoading } = useSWR(
    `/api/produk/${query.produk}`,
    fetcher
  );

  return (
    <div>
      <h1>Halaman Produk</h1>
      <p>Produk: {query.produk}</p>
    </div>
  );
```

2	Pada file produk.ts pada folder pages/api di rename menjadi [[...product]].ts 

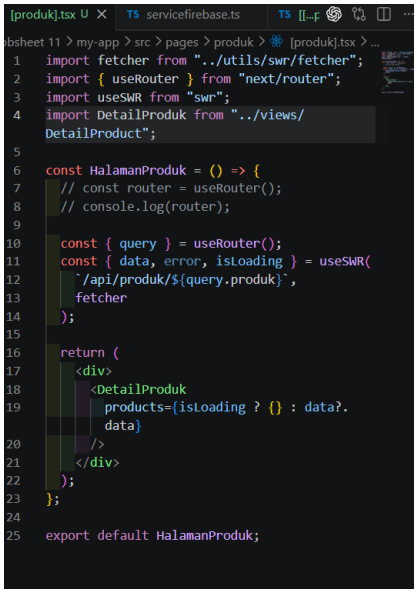
3	Modifikasi file servicefirebase.ts
	
	import {
	getFirestore,
	collection,
	getDocs,
	Firestore,
	getDoc,
	doc
	} from "firebase/firestore";
	import app from "./firebase";
	
	const db = getFirestore(app);
	
	export async function retrieveProducts
	(collectionName: string) {
	const snapshot = await getDocs(collection(db,
	collectionName));
	const data = snapshot.docs.map((doc) => ({
	id: doc.id,
	...doc.data(),
	}));
	return data;
	}
	
	export async function retrieveDataByID(
	collectionName: string,
	id: string
	) {
	const snapshot = await getDoc(doc(db,
	collectionName, id));
	const data = snapshot.data();
	return data;
	}

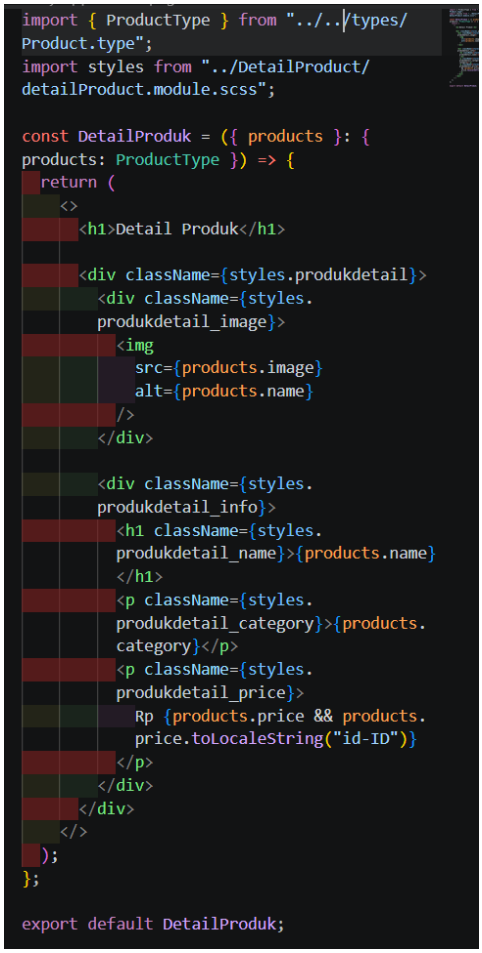
4	Modifikasi file [[...produk]].ts Mengimplementasikan CSR untuk produk

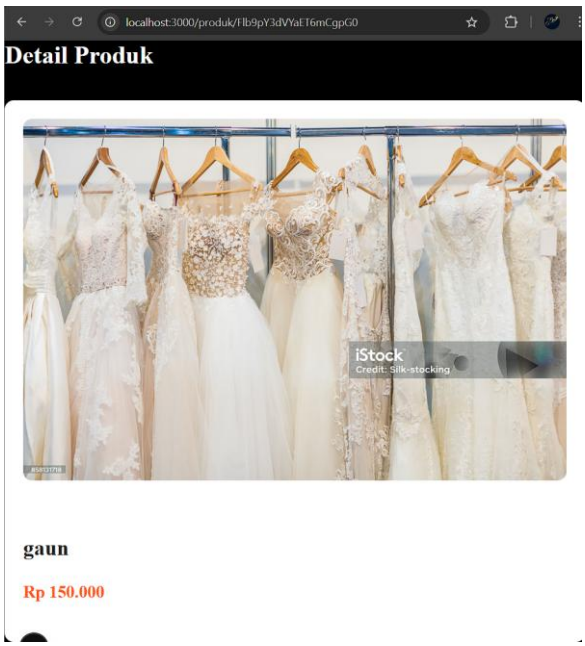
5	Jalankan browser http://localhost:3000/api/produk/3FwB5UeuLVBpQy2umXHD
6	Jalankan alamat url http://localhost:3000/api/produk/123 
7	Buat file dengan nama index.tsx pada folder views/DetailProduct selain itu buat juga file dengan nama detailProduct.module.scss 
8	Modifikasi detailProduct.module.scss

9	Modifikasi index.tsx pada folder DetailProduct

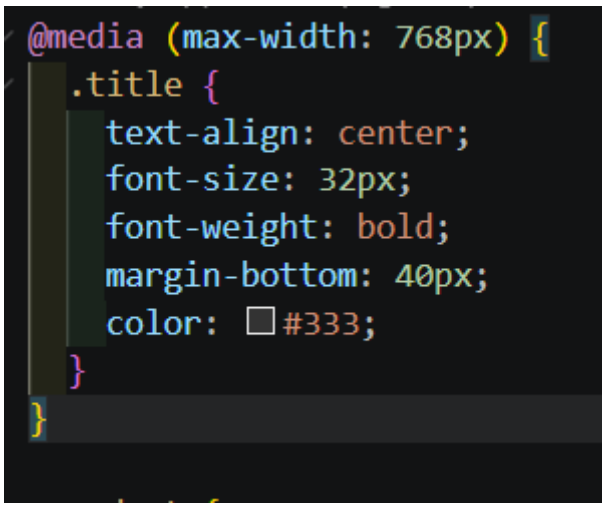
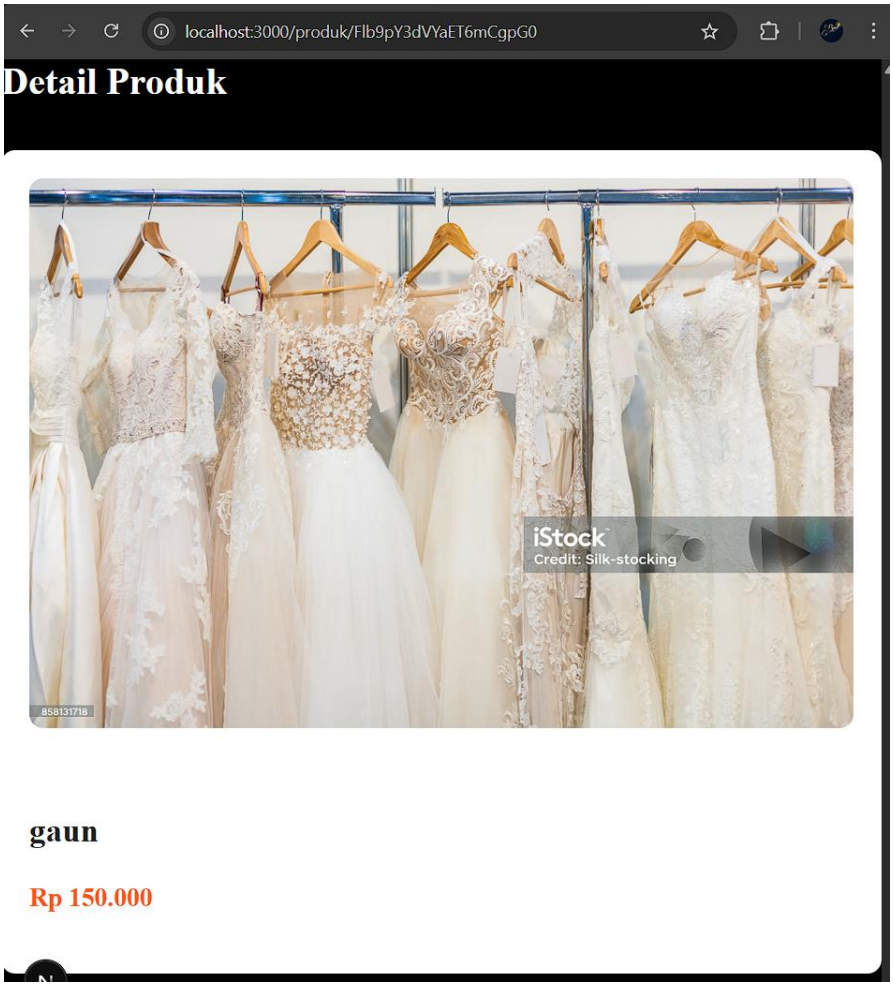
```
import { ProductType } from "../../types/
Product.type";
import styles from "./detailProduct.module.
scss";

const DetailProduk = ({ products }: {
products: ProductType }) => {
  const price =
    typeof products.price === "number"
      ? products.price
      : Number(products.price);

  return (
    <div className={styles.produkDetail}>
      <div className={styles.image}>
        <img src={products.image} alt=
        {products.name} />
      </div>

      <div className={styles.info}>
        <h1 className={styles.name}>
        {products.name}</h1>
        <p className={styles.category}>
        {products.category}</p>
        <p className={styles.price}>Rp
        {price.toLocaleString("id-ID")}</p>
      </div>
    </div>
  );
};

export default DetailProduk;
```

10	Modifikasi file [product].tsx  [produk].tsx U × TS servicefirebase.ts TS [[...p bsheet 11 > my-app > src > pages > produk > [produk].tsx > ... 1 import fetcher from "../utils/swr/fetcher"; 2 import { useRouter } from "next/router"; 3 import useSWR from "swr"; 4 import DetailProduk from "../views/DetailProduct"; 5 6 const HalamanProduk = () => { 7 // const router = useRouter(); 8 // console.log(router); 9 10 const { query } = useRouter(); 11 const { data, error, isLoading } = useSWR(12 `/api/produk/${query.produk}`, 13 fetcher 14); 15 16 return (17 18 <DetailProduk 19 products={isLoading ? {} : data?.data} 20 /> 21 22); 23 }; 24 25 export default HalamanProduk;

11	Modifikasi index.tsx pada folder views/detailProduct line 16
	 
	import { ProductType } from "../../types/Product.type";
	import styles from "../DetailProduct/detailProduct.module.scss";
	
	const DetailProduk = ({ products }: {
	products: ProductType }) => {
	return (
	<>
	<h1>Detail Produk</h1>
	
	<img
	src={products.image}
	alt={products.name}
	/>
	
	<h1 className={styles.produkdetail_name}>{products.name}
	</h1>
	 {products.category}
	
	Rp {products.price && products.price.toLocaleString("id-ID")}
	
	</>
	);
	};
	
	export default DetailProduk;

12	Jalankan browser http://localhost:3000/produk/ saat produk diklik maka akan muncul detailProduk http://localhost:3000/produk/pAWIT99SWmVbVrNm49ml 
13	Agar tulisan detail produk ditengah maka modifikasi file detailProduct.module.scss line 103-108 dan file index.tsx tambahkan code pada line 7,8 dan 22

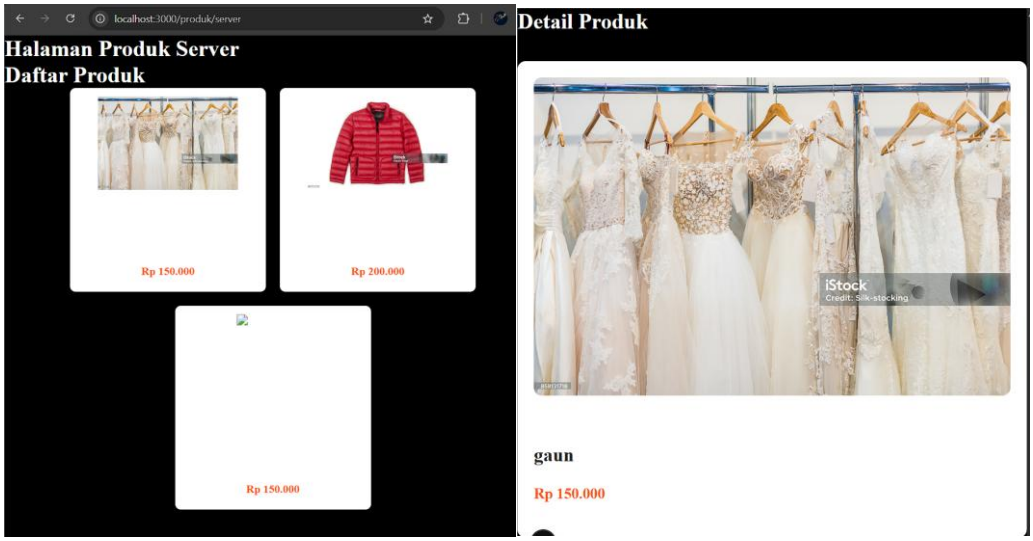
```
import { ProductType } from "../../types/Product.
type";
import styles from "../DetailProduct/
detailProduct.module.scss";

const DetailProduk = ({ products }: { products:
ProductType }) => {
  return (
    <>
      <h1 className={styles.title}>Detail Produk</
      h1>

      <div className={styles.produkdetail}>
        <div className={styles.produkdetail_image}
        >
          <img
            src={products.image}
            alt={products.name}
          />
        </div>

        <div className={styles.produkdetail_info}>
          <h1 className={styles.produkdetail_name}
          >{products.name}</h1>
          <p className={styles.
          produkdetail_category}>{products.
          category}</p>
          <p className={styles.produkdetail_price}
          >
            Rp {products.price && products.price.
            toLocaleString("id-ID")}
          </p>
        </div>
      </div>
    </>
  );
};

export default DetailProduk;
```

	
14	Sehingga hasilnya seperti berikut  Berhasil menambahkan detail setiap product

Langkah 3 – Implementasi SSR

Langkah	Jawaban/Deskripsi
1	Modifikasi [produk].tsx pada folder src/pages/produk dan comment line 9 sampai 20 dikarena kita akan menggunakan metode SSR. Tambahkan beberapa kode untuk SSR

```
import fetcher from "../utils/swr/fetcher";
import { useRouter } from "next/router";
import useSWR from "swr";
import DetailProduk from "../views/DetailProduct";
import { ProductType } from "../types/Product.
type";

const HalamanProduk = ({ product }: { product:
ProductType }) => {
  // (digunakan client-side rendering)
  // const router = useRouter();
  // console.log(router);

  // const { query } = useRouter();
  // const { data, error, isLoading } = useSWR(
  //   `/api/produk/${query.produk}`,
  //   fetcher
  // );

  // return (
  //   <div>
  //     <DetailProduk products={isLoading ? {} :
  data?.data} />
  //   </div>
  // );

  return (
    <div>
      <DetailProduk products={product} />
    </div>
  );
};

export default HalamanProduk;

// Fungsi getServerSideProps akan dipanggil
setiap kali halaman ini diakses,
// dan akan mengambil data dari API sebelum
merender halaman.
// (digunakan server-side rendering)
export async function getServerSideProps({
params }: { params: { produk: string } }) {
```

2	Jalankan browser http://localhost:3000/produk/server 

Langkah 4 – Implementasi Static Site Generation (Dynamic SSG)

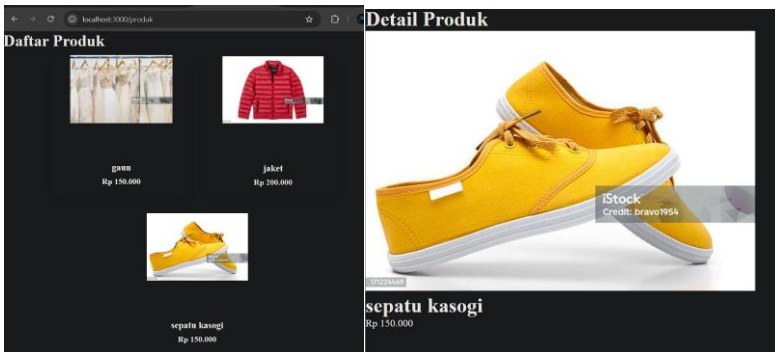
Langkah	Jawaban/Deskripsi
1	Buka file [produk].tsx dan modifikasi seperti berikut

```
import fetcher from "../utils/swr/fetcher";
import { useRouter } from "next/router";
import useSWR from "swr";
import DetailProduk from "../views/DetailProduct";
import { ProductType } from "../types/Product.
type";

const HalamanProduk = ({ product }: { product:
ProductType }) => {
  // (digunakan client-side rendering)
  // const router = useRouter();

  return (
    <div>
      <DetailProduk products={product} />
    </div>
  );
};

export default HalamanProduk;

// ==============================================
// (digunakan server-side rendering)
// ==============================================
// export async function getServerSideProps({
params }: { params: { produk: string } }) {
//   const res = await fetch(`http://
localhost:3000/api/produk/${params.produk}`);
//   const response = await res.json();

//   return {
//     props: {
//       product: response.data,
//     },
//   };
// }

// ==============================================
// (digunakan static-site generation)
// ==============================================
export async function getStaticPaths() {
  const res = await fetch("http://localhost:3000/
```

2	Buka file index.tsx pada folder src/views/DetailProduct dan modifikasi pada line 11

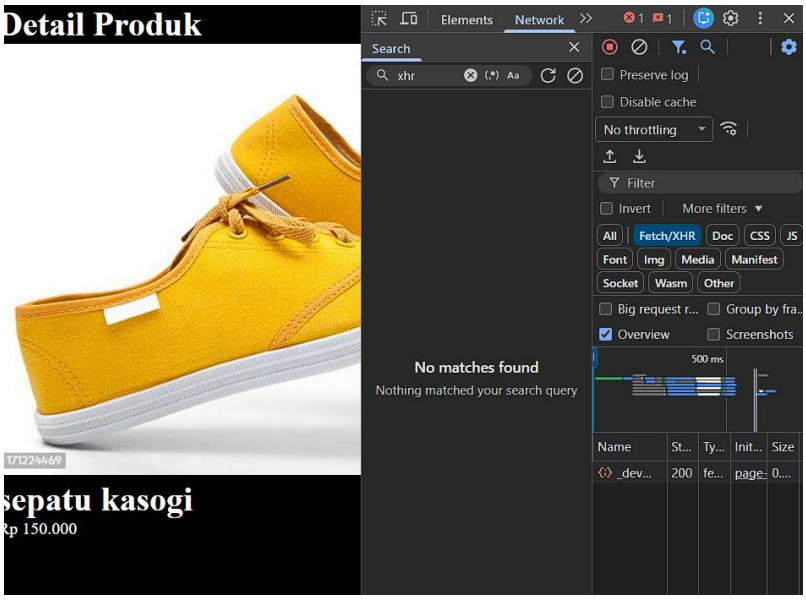
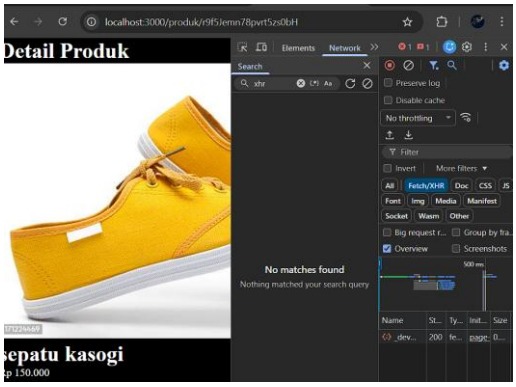
```
import { ProductType } from "../../types/Product.
type";
import styles from "../DetailProduct/
detailProduct.module.scss";

const DetailProduk = ({ products }: { products:
ProductType }) => {
  return (
    <>
      <h1 className={styles.title}>Detail Produk</
      h1>

      <div className={styles.produkdetail}>
        <div className={styles.produkdetail_image}
        >
          <img
            src={products.image}
            alt={products.name}
          />
        </div>

        <div className={styles.produkdetail_info}>
          <h1 className={styles.produkdetail_name}
          >{products.name}</h1>
          <p className={styles.
          produkdetail_category}>{products.
          category}</p>
          <p className={styles.produkdetail_price}
          >
            Rp {products.price && products.price.
            toLocaleString("id-ID")}
          </p>
        </div>
      </div>
    </>
  );
};

export default DetailProduk;
```

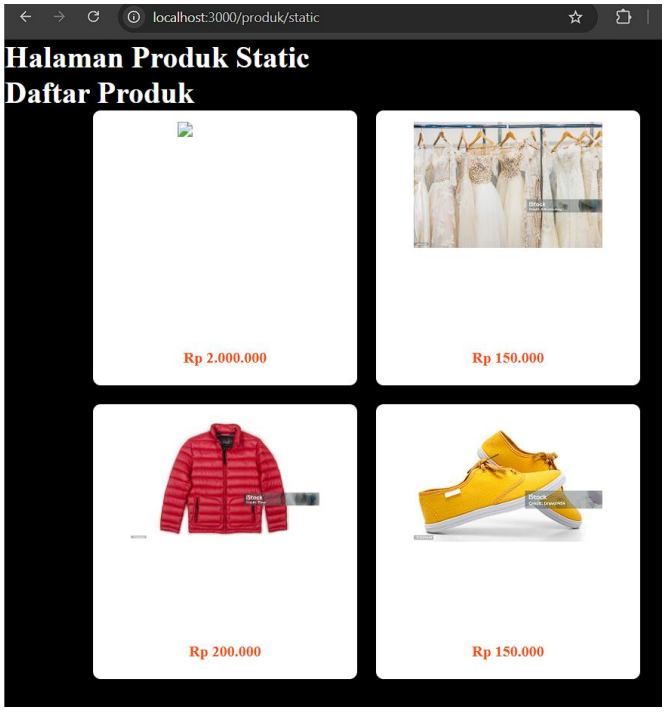
3	Jalankan browser http://localhost:3000/produk



Berhasil menambahkan SSG untuk detail produk

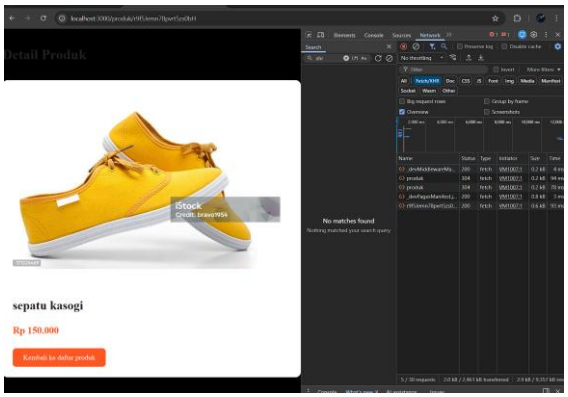
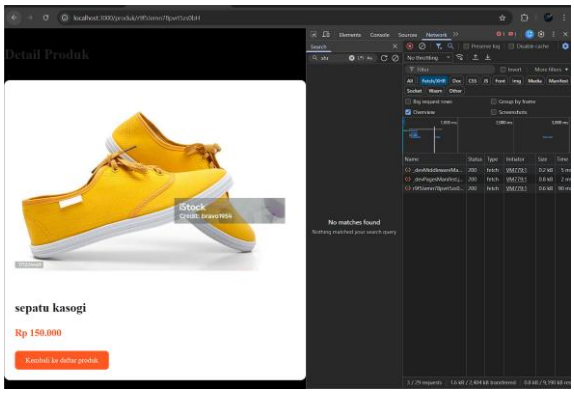
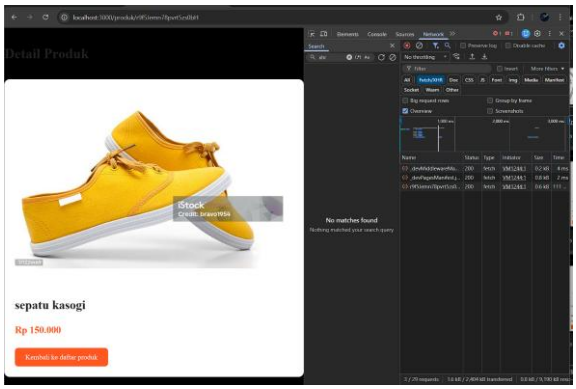
D. Pengujian

Langkah	Jawaban/Deskripsi
1	Uji 1 – CSR • Refresh halaman detail • Perhatikan loading • Periksa Network → XHR → API request terlihat 
2	Uji 2 – SSR • Refresh halaman detail • Tidak ada loading • Periksa Network → tidak terlihat fetch detail 

3	Uji 3 – SSG (Lakukan seperti langkah sebelumya pada Jobsheet 10) 1. Jalankan: o npm run build o npm run start 2. Tambahkan produk baru di database. 3. Buka halaman detail produk baru: 4. Tidak muncul. Build ulang: o npm run build o npm run start Baru muncul 

E. Tugas Praktikum

Langkah	Jawaban/Deskripsi
1	

	Implementasikan halaman detail dengan: o CSR  o SSR  o SSG 
2	

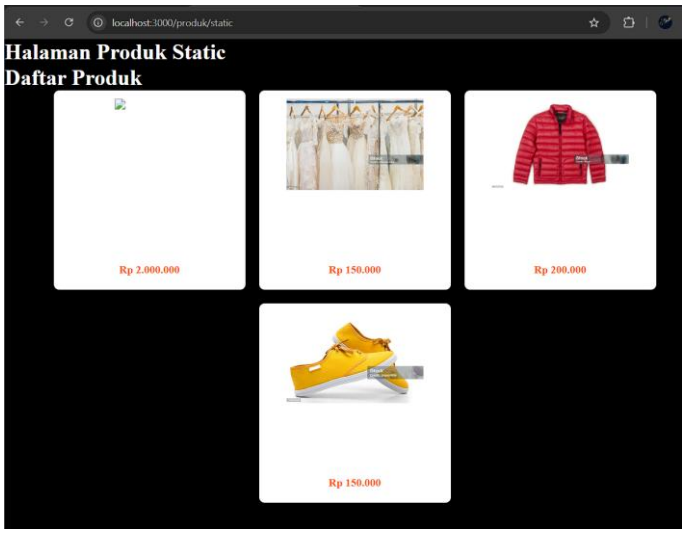
Buat tabel perbandingan:

Aspek	CSR	SSR	SSG
Loading	Ada	Tidak ada	Tidak ada
Build Required	Tudaj	Tidak	Ya dengan NPM run build
SEO	Kurang	Baik	Sangat baik
Perubahan Data	Labngsung terupdate	Selalu fresh setiap request	Harus rebuild dulu

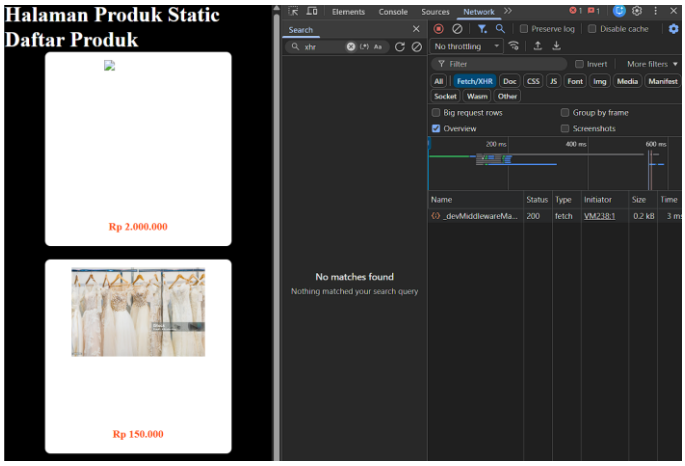
3

Dokumentasikan:

Screenshot:



Network Tab:



	Build Result: `Import trace for requested module: ./src/pages/auth/register/Register.module.scss.webpack[javascript/auto]!=!./node_modules/next/dist/build/webpack/loaders/css-loader/src/index.js??ruleSet[1].rules[9].oneOf[9].use[1]!./node_modules/next/dist/build/webpack/loaders/postcss-loader/src/index.js??ruleSet[1].rules[9].oneOf[9].use[2]!./node_modules/next/dist/build/webpack/loaders/resolve-url-loader/index.js??ruleSet[1].rules[9].oneOf[9].use[3]!./node_modules/next/dist/compiled/sass-loader/cjs.js??ruleSet[1].rules[9].oneOf[9].use[4]!./src/pages/auth/register/Register.module.scss ./src/pages/auth/register/Register.module.scss` `✓ Compiled successfully in 6.4s` `✓ Collecting page data using 11 workers in 4.7s`

F. Pertanyaan Analisis

Langkah	Jawaban/Deskripsi
1	Mengapa getStaticPaths wajib pada dynamic SSG? getStaticPaths wajib digunakan pada dynamic SSG karena Next.js perlu mengetahui daftar path mana saja yang harus dibuat secara statis saat proses build
2	Mengapa CSR membutuhkan loading state? CSR membutuhkan loading state karena data baru di fetch di sisi client setelah halaman pertama kali dimuat di browser
3	Mengapa SSG tidak menampilkan produk baru tanpa build ulang? SSG tidak menampilkan produk baru tanpa build ulang karena halaman detail sudah di generate secara statis pada saat build time
4	Mana metode terbaik untuk halaman detail e-commerce? Untuk halaman detail e-commerce, metode terbaik biasanya adalah SSR karena menggabungkan performa baik dengan data yang selalu diperbarui
5	Apa risiko menggunakan SSG untuk produk yang sering berubah? Risiko menggunakan SSG untuk produk yang sering berubah adalah data yang ditampilkan menjadi outdated sampai proses build dijalankan ulang